



VISIONONLY

Lenguajes y Gramáticas Formales

“Mantenlo Simple”

Naleska Carrera
Ana Santamaria
Genesis Moya
Carlos Martínez

FUNDAMENTOS Y JERARQUÍA DE LAS GRAMÁTICAS FORMALES.

En la informática teórica, un lenguaje de programación no es un conjunto arbitrario de texto plano; es un objeto matemático estructurado.

$$G=(V,\Sigma,S,P)$$

ELEMENTOS FUNDAMENTALES

- **V**, el conjunto finito de símbolos no terminales o variables sintácticas.
- **Σ** , el alfabeto de símbolos terminales que formarán las cadenas finales. Es una regla de oro que se cumpla la propiedad de exclusión mutua.
- **S**, el axioma o símbolo inicial.
- **P**, el conjunto finito de reglas de producción o reescritura de la forma $\alpha \rightarrow \beta$.

LA JERARQUÍA DE CHOMSKY

TIPO 0 | SIN RESTRICCIONES

- Potencia de la Máquina de Turing.
- No se usan para sintaxis por ser **no decidibles**.
- Restricción de producto: $\alpha A \beta \rightarrow \delta$

TIPO 1 | SENSIBLES AL CONTEXTO

- Autómatas Linealmente Acotados.
- Asociado al Análisis Semántico: validación de declaraciones y entornos locales.
- Restricción de producto: $\alpha A \beta \rightarrow \alpha \gamma \beta$

TIPO 2: GRAMÁTICAS LIBRES DE CONTEXTO

- El núcleo de la programación comercial.
- Permite modelar recursividad y estructuras jerárquicas anidadas.
- Máquina: Autómata con Pila.
- Restricción de producto: $A \rightarrow \gamma$

TIPO 3: EL MOTOR DEL ANÁLISIS LÉXICO

- El nivel más restrictivo.
- Sus producciones son estrictamente lineales.
- Agrupa caracteres en fichas o tokens mediante
- Autómatas Finitos.
- Restricción de producto: $A \rightarrow a B | a$

DEMOSTRACIÓN PRÁCTICA EN NOTACIÓN BNF

03. Tipo 3 (Regular)

Las gramáticas regulares definen patrones lineales simples.

- `<Identificador> ::= <Letra>`
- `<Identificador> ::= <Letra> <Resto>`
- `<Resto> ::= <Letra> | <Digito> | <Letra> <Resto> | <Digito> <Resto>`

02. Tipo 3 (Regular)

Las gramáticas libres de contexto manejan estructuras jerárquicas y anidadas.

- `<S> ::= ( )`
- `<S> ::= ( <S> )`
- `<S> ::= <S> <S>`

DEMOSTRACIÓN PRÁCTICA EN NOTACIÓN BNF

01. Tipo 1 (Sensible al Contexto)

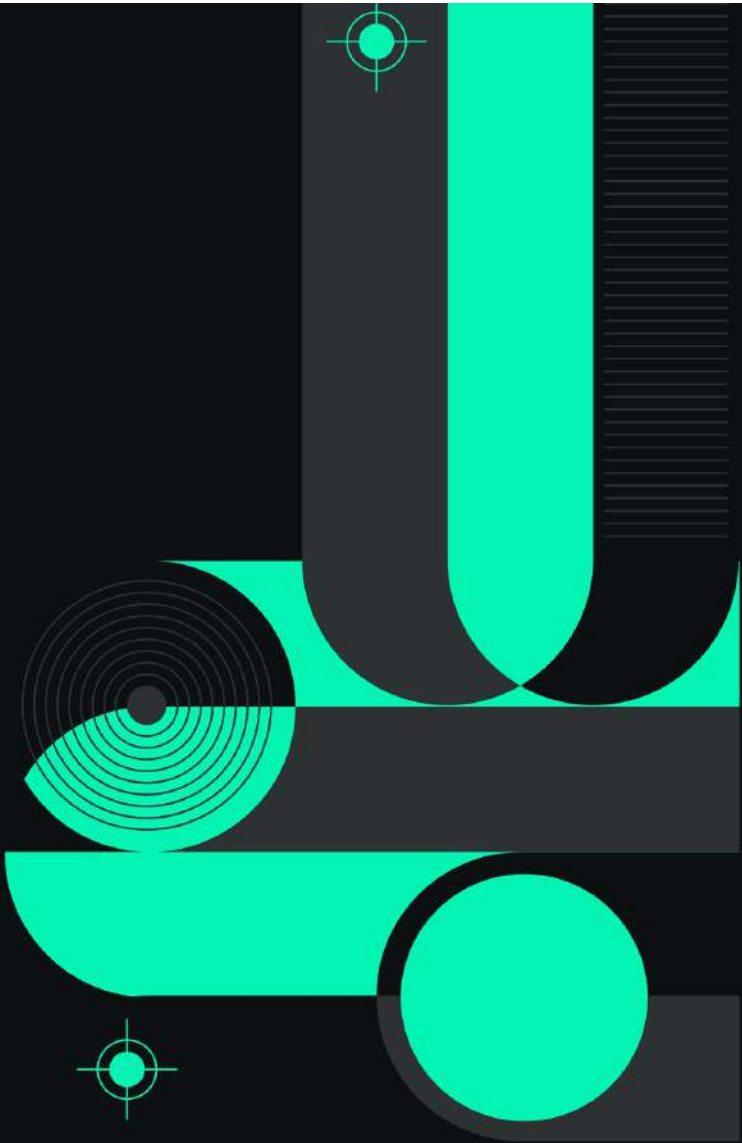
Aquí entramos en estructuras donde las reglas dependen estrictamente del entorno.

- $\langle S \rangle ::= a \ b \ c$
- $\langle S \rangle ::= a \ \langle S \rangle \ \langle B \rangle \ c$
- $\langle B \rangle \ b ::= b \ b$
- $\langle B \rangle \ c ::= b \ c$

00. Tipo 0 (Sin Restricciones)

Las gramáticas de Tipo 0 representan la libertad absoluta de reescritura de cadenas.

- $\langle A \rangle \ \langle B \rangle ::= \langle C \rangle$
- $\langle C \rangle \ \langle D \rangle ::= \langle D \rangle \ \langle A \rangle$
- $\langle S \rangle ::= \langle A \rangle \ \langle B \rangle \ \langle D \rangle$

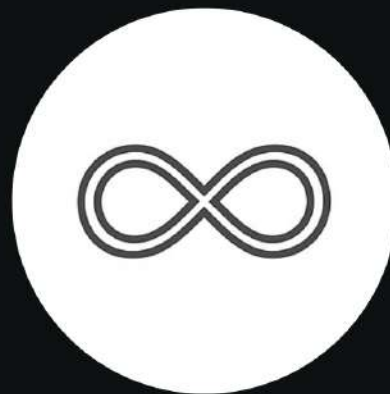


HIGIENE Y OPTIMIZACIÓN DE GRAMÁTICAS FORMALES

GENESIS MOYA

¿POR QUÉ ES NECESARIA LA HIGIENE DE UNA GRAMÁTICA?

"La higiene no es un mero ejercicio académico, sino una práctica de ingeniería esencial"



GENESIS MOYA

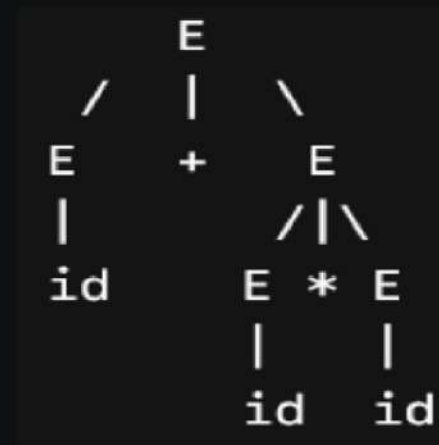
GRAMÁTICA AMBIGUA

Una gramática es ambigua si genera más de un árbol de derivación para una misma cadena.

Árbol Izquierdo: Agrupa primero la suma (incorrecto).



Árbol Derecho: Agrupa primero la multiplicación (correcto).



SOLUCIÓN: ESTRATIFICACIÓN Y PARSING EN PYTHON

Métodos `parse_E()`

```
1  def parse_E(self):
2      """
3      Regla Expresión (E -> T + E | T)
4      Maneja la suma (Menor precedencia).
5      """
6      nodo_izq = self.parse_T()
7
8      # Si encontramos un signo '+', resolvemos la estructura
       jerárquica
9      if self.obtener_token_actual() == '+':
10         op = self.obtener_token_actual()
11         self.pos += 1 # Consumir el '+'
12         nodo_der = self.parse_E()
13         return f"[{nodo_izq} {op} {nodo_der}]"
14
15     return nodo_izq
```

GENESIS MOYA

SOLUCIÓN: ESTRATIFICACIÓN Y PARSING EN PYTHON

parse_T()

```
1 def parse_T(self):
2     """
3     Regla Término (T -> F * T | F)
4     Maneja la multiplicación (Mayor precedencia).
5     """
6     nodo_izq = self.parse_F()
7
8     # Si encontramos un signo '*', se agrupa primero aquí
9     if self.obtener_token_actual() == '*':
10         op = self.obtener_token_actual()
11         self.pos += 1 # Consumir el '*'
12         nodo_der = self.parse_T()
13         return f"[{nodo_izq} {op} {nodo_der}]"
14
15     return nodo_izq
```

GENESIS MOYA

SOLUCIÓN: ESTRATIFICACIÓN Y PARSING EN PYTHON

parse_F()

```
1 def parse_F(self):
2     """
3     Regla Factor (F -> id)
4     Maneja los elementos terminales (identificadores o variables
5     ).
6     """
7     token = self.obtener_token_actual()
8     if token == 'id':
9         self.pos += 1 # Consumir el 'id'
10        return "id"
11
12    raise SyntaxError(f"Error Sintáctico: Se esperaba 'id', se
13    obtuvo '{token}'")
```

GENESIS MOYA

RESULTADO DEL SCRIPT

```
Cadena de entrada: id + id * id
```

```
--- Árbol Sintáctico Único Generado ---  
[id + [id * id]]
```



GENESIS MOYA

RECURSIVIDAD POR LA IZQUIERDA

El problema: Reglas de la forma $A \rightarrow Aa \mid \beta$. Provoca bucles infinitos en parsers descendentes.

Caso Practico:

Lista de variables
separadas por
comas: $L \rightarrow L, id \mid id$

Algoritmo matemático de eliminación paso a paso:

- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \epsilon$

ALGORITMO DE ELIMINACIÓN EN PYTHON

Diagrama lógico o fragmento del script (funciones de detección de prefijo recursivo, aislamiento de alphas y betas y reconstrucción de reglas).

Salida Final:

- $L \rightarrow id \ L'$
- $L' \rightarrow , id \ L' \mid \epsilon$

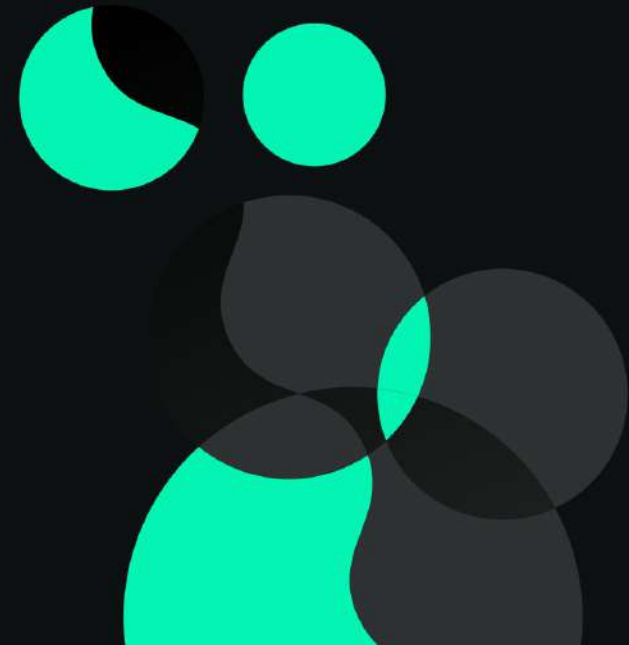


FACTORIZACIÓN POR LA IZQUIERDA

Producciones que comparten un prefijo común (a), impidiendo la predicción determinista.

Caso Practico (Dangling Else):

- $S \rightarrow \text{if } E \text{ then } S$
- $S \rightarrow \text{if } E \text{ then } S \text{ else } S$
- $S \rightarrow \text{assignment}$



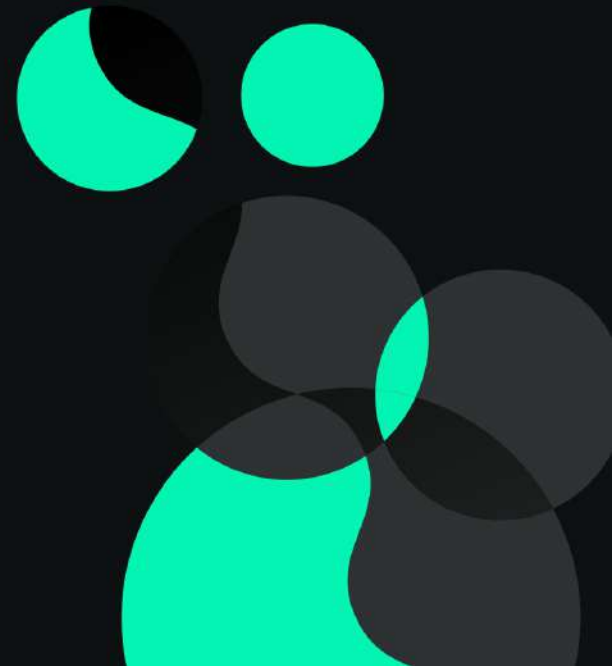
SOLUCIÓN ALGORÍTMICA CON PYTHON

Uso de `defaultdict(list)` para agrupar dinámicamente por tokens.

Gramática Resultante Optimizada generada:

- $S \rightarrow \text{if } E \text{ then } S \ S' \mid \text{assignment}$
- $S' \rightarrow \text{else } S \mid \epsilon$

GENESIS MOYA





LENGUAJE & COMPILADORES

Derivación y Modelado.

(Caso Práctico: Genoma).

ANA SANTAMARIA



DIAGRAMA DE LIBRE CONTEXTO (GLC)

En términos prácticos, una GLC actúa como un conjunto estricto de reglas de sustitución. Comienza con un axioma o símbolo inicial y, mediante reemplazos sucesivos basados en estas reglas, genera secuencias de símbolos "terminales" (las unidades indivisibles del lenguaje, como palabras en un texto, o instrucciones de dibujo en nuestro caso genómico). Esta capacidad estructurada permite modelar desde la sintaxis de bucles y condicionales en compiladores informáticos, hasta las complejas jerarquías de ramificación en organismos biológicos, como se evidencia en los Sistemas de Lindenmayer o L-System.

L-SYSTEM

Fueron introducidos en 1968 por Aristid Lindenmayer, un biólogo teórico y botánico húngaro. Su objetivo original no era crear gráficos por computadora, sino desarrollar un modelo matemático formal para estudiar el crecimiento, el desarrollo y los patrones de ramificación de las plantas, algas y hongos celular por celular.

REESCRITURA PARALELA

En cada iteración (que representa un ciclo de tiempo o de crecimiento), todas las letras de la cadena son evaluadas y reemplazadas simultáneamente por sus reglas de producción. Esto imita la biología real: las células de un árbol no crecen una por una de forma secuencial, sino que todas se dividen al mismo tiempo.

MEMORIA ESPACIAL

- [(Push): Guarda la posición (x, y) y el ángulo actual del cursor en una memoria temporal

] (Pop): Saca la última posición guardada de la pila y teletransporta el cursor allí, restaurando su ángulo.

NATURALEZA FRACTAL

Debido a su recursividad profunda, unas pocas iteraciones generan cadenas con miles de caracteres. Un simple axioma (estado inicial) de dos letras puede convertirse en una estructura de asombrosa complejidad geométrica y autosimilitud.

GENOMA: PARALELISMO Y CRECIMIENTO

Concepto Biológico	Concepto Computacional (Nuestro Modelo)
ADN	Alfabeto $\Sigma = \{a, c, g, t\}$
Código Genético	Cadena derivada (ej. a c c a...)
Crecimiento Celular	Ejecución del script (Motor de trazado)
Organismo Físico	Figura geométrica renderizada

SUBCONJUNTO SIMPLIFICADO

Para nuestro caso práctico,
modelamos una jugada completa
bajo la siguiente estructura:

Turno + Blanco + Espacio + Negro

01

02

03

04

Reglas de Producción

- Número seguido de punto.
- Pieza opcional + Coordenada.
- Espacio obligatorio.
- Respuesta de las negras.

LA EXPRESIÓN REGULAR (REGEX)

Esta expresión define el patrón léxico que el motor de reconocimiento debe validar antes de permitir el paso de la cadena al proceso de sintaxis.

```
[1-9][0-9]*\.[KQRBN]?[a-h][1-8] [KQRBN]?[a-h][1-8]
```



Diseño del Autómata (AFD)

LÓGICA DE CONTROL

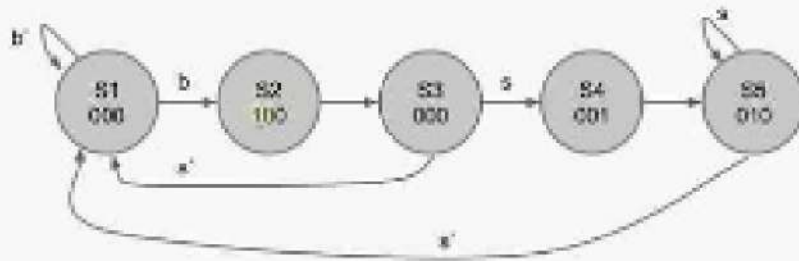
El autómata se desplaza por 11 estados posibles ($q_0 - q_{10}$). Cada entrada genera una transición única.

ESTADO DE ERROR (QE):

Cualquier carácter fuera del manual de reglas colapsa el sistema, garantizando que solo jugadas perfectas sean aceptadas.

FSM Diagram

Outputs = d1d21



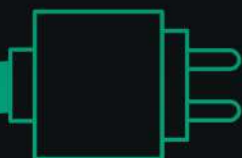


TABLA DE TRANSISIONES

Estado	Entrada	Destino	Descripción
q0	[1-9]	q1	Inicio: Número de jugada
q1	[0-9] .	q1 q2	Ciclo de dígitos y punto
q2	[Pieza] [Columna]	q3 q4	Movimiento Blancas
q4	[Fila]	q5	Fin Blancas
q5	''	q6	Espacio de separación
q6	[Pieza] [Columna]	q7 q8	Movimiento Negras
q8	[Fila]	q10	Estado Final de Aceptación

IMPLEMENTACIÓN EN PYTHON

El código utiliza una estructura IF-ELIF para simular el comportamiento del AFD físico.

Se lee la cadena carácter por carácter, moviendo la variable "estado" según la tabla de transiciones definida.

```
3  def analizador_lexico_pgn(cadena):
4      # Definición de conjuntos de símbolos del alfabeto
5      piezas = {'K', 'Q', 'R', 'B', 'N'}
6      columnas = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'}
7      filas = {'1', '2', '3', '4', '5', '6', '7', '8'}
8      digitos_no_cero = {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
9      todos_los_digitos = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'}
10
11     # Estado inicial
12     estado = "q0"
```

```

13
14 print(f"Analizando cadena: '{cadena}'")
15 print(f"[{estado}] -> Inicio del analisis")
16
17 for i, caracter in enumerate(cadena):
18     estado_anterior = estado
19
20     #Transiciones del AFD
21     if estado == "q0":
22         if caracter in digitos_no_cero:
23             estado = "q1"
24         else:
25             estado = "qe"
26
27     elif estado == "q1":
28         if caracter in todos_los_digitos:
29             estado = "q1"
30         elif caracter == ".":
31             estado = "q2"
32         else:
33             estado = "qe"
34
35     elif estado == "q2":
36         if caracter in piezas:
37             estado = "q3"
38         elif caracter in columnas:
39             estado = "q4"
40         else:
41             estado = "qe"
42
43     elif estado == "q3":
44         if caracter in columnas:
45             estado = "q4"
46         else:

```

```

48
49     elif estado == "q4":
50         if caracter in filas:
51             estado = "q5"
52         else:
53             estado = "qe"
54
55     elif estado == "q5":
56         if caracter == " ":
57             estado = "q6"
58         else:
59             estado = "qe"
60
61     elif estado == "q6":
62         if caracter in piezas:
63             estado = "q7"
64         elif caracter in columnas:
65             estado = "q8"
66         else:
67             estado = "qe"
68
69     elif estado == "q7":
70         if caracter in columnas:
71             estado = "q8"
72         else:
73             estado = "qe"
74
75     elif estado == "q8":
76         if caracter in filas:
77             estado = "q10"
78         else:
79             estado = "qe"
80
81     elif estado == "q10":

```

```
# Si cae en estado de error, rompemos el ciclo (Early exit)
if estado == "qe":
    print(f" Error en carácter '{caracter}' (posición {i}): Transición inválida desde {estado_anterior}.")
    return False, f"Rechazada: Error sintáctico en la posición {i} ('{caracter}')"

print(f" Tránsito: '{caracter}' -> [{estado}]")

# --- Verificación del Estado Final ---
if estado == "q10":
    return True, "¡Cadena ACEPTADA! Sintaxis de jugada PGN válida."
else:
    return False, f"Rechazada: La cadena terminó en el estado incompleto [{estado}]."
```



```
100 # Casos de Prueba para Demostración
101 if __name__ == "__main__":
102     pruebas = [
103         "1.e4 e5",      # Válida (Peones)
104         "12.Nf3 d6",    # Válida (Pieza blanca, Peon negro, Nro multiple digito)
105         "3.Qd1 Qh5",    # Válida (Ambas piezas mayores)
106         "1.e9 e5",      # Inválida (Fila 9 no existe)
107         "1.x4 e5",      # Inválida (Columna x no existe)
108         "1.e4",         # Inválida (Falta el movimiento de las negras)
109         "01.e4 e5"      # Inválida (No puede empezar con cero)
110     ]
111
112     print("=== EJECUCIÓN DEL COMPILADOR (ANALIZADOR PGN) ===\n")
113     for cadena in pruebas:
114         es_valida, mensaje = analizador_lexico_pgn(cadena)
115         print(f"Resultado final: {mensaje}")
116         print("-" * 50)
```

SALIDA DEL ANALIZADOR DE EXPRESIONES PGN

```
PS C:\Users\PC1\OneDrive\Documentos\GitHub\LC-VisionLy\Tema 3\Actividad 2.4 PGN Ajedrez> py1
```

```
-----  
Analizando cadena: '3.Qd1 Qh5'
```

```
[q0] -> Inicio del analisis
```

```
Tránsito: '3' -> [q1]
```

```
Tránsito: '.' -> [q2]
```

```
Tránsito: 'Q' -> [q3]
```

```
Tránsito: 'd' -> [q4]
```

```
Tránsito: '1' -> [q5]
```

```
Tránsito: ' ' -> [q6]
```

```
Tránsito: 'Q' -> [q7]
```

```
Tránsito: 'h' -> [q8]
```

```
Tránsito: '5' -> [q10]
```

```
Resultado final: ¡Cadena ACEPTADA! Sintaxis de jugada PGN válida.
```

```
-----  
Analizando cadena: '1.e9 e5'
```

```
[q0] -> Inicio del analisis
```

```
Tránsito: '1' -> [q1]
```

```
Tránsito: '.' -> [q2]
```

```
Tránsito: 'e' -> [q4]
```

```
Error en carácter '9' (posición 3): Transición inválida desde q4.
```

```
Resultado final: Rechazada: Error sintáctico en la posición 3 ('9')
```

```
-----  
Analizando cadena: '1.x4 e5'
```

```
[q0] -> Inicio del analisis
```

```
Tránsito: '1' -> [q1]
```

```
Tránsito: '.' -> [q2]
```

```
Error en carácter 'x' (posición 2): Transición inválida desde q2.
```

```
Resultado final: Rechazada: Error sintáctico en la posición 2 ('x')
```

```
-----  
Analizando cadena: '1.e4'
```

```
[q0] -> Inicio del analisis
```

```
Tránsito: '1' -> [q1]
```

```
Tránsito: '.' -> [q2]
```

```
Tránsito: 'e' -> [q4]
```

```
Tránsito: '4' -> [q5]
```

```
Resultado final: Rechazada: La cadena terminó en el estado incompleto [q5].
```

```
-----  
Analizando cadena: '01.e4 e5'
```

```
[q0] -> Inicio del analisis
```

```
Error en carácter '0' (posición 0): Transición inválida desde q0.
```

```
Resultado final: Rechazada: Error sintáctico en la posición 0 ('0')
```

```
Analizando cadena: '1.e4 e5'
```

```
[q0] -> Inicio del analisis
```

```
Tránsito: '1' -> [q1]
```

```
Tránsito: '.' -> [q2]
```

```
Tránsito: 'e' -> [q4]
```

```
Tránsito: '4' -> [q5]
```

```
Tránsito: ' ' -> [q6]
```

```
Tránsito: 'e' -> [q8]
```

```
Tránsito: '5' -> [q10]
```

```
Resultado final: ¡Cadena ACEPTADA! Sintaxis de jugada PGN válida.
```

```
-----  
Analizando cadena: '12.Nf3 d6'
```

```
[q0] -> Inicio del analisis
```

```
Tránsito: '1' -> [q1]
```

```
Tránsito: '2' -> [q1]
```

```
Tránsito: '.' -> [q2]
```

```
Tránsito: 'N' -> [q3]
```

```
Tránsito: 'f' -> [q4]
```

```
Tránsito: '3' -> [q5]
```

```
Tránsito: ' ' -> [q6]
```

```
Tránsito: 'd' -> [q8]
```

```
Tránsito: '6' -> [q10]
```

```
Resultado final: ¡Cadena ACEPTADA! Sintaxis de jugada PGN válida.
```

¡Gracias!

